

JavaScript Async & Await Notes

What is Async & Await?

Async and Await are keywords introduced in JavaScript to make asynchronous code look more readable and sequential like synchronous code. Internally, async/await works on top of Promises.

Keyword	Purpose
async	Makes a function return a Promise
await	Pauses async function execution until Promise resolves

Promise Chaining Example

```
fetch(url) .then(response => response.json()) .then(data => console.log(data));
```

Async/Await Example

```
async function getData() {  
  const response = await fetch(url);  
  const data = await response.json();  
  console.log(data);  
}
```

Important Concepts

1. Async function always returns a Promise.
2. Await only works inside async functions.
3. Await pauses only the async function, not the entire JavaScript engine.
4. Event loop continues executing other code while async function waits.
5. Async/Await improves workflow readability and maintainability in large applications.

Real-Time Example

Student Dashboard Loading Flow:

1. Login Student
2. Fetch Student Details
3. Fetch Attendance
4. Fetch Notifications
5. Render Dashboard

Each step depends on previous API response. Async/Await makes this workflow easier to read and maintain.

Error Handling using try...catch

```
async function loadData() {  
  try {  
    const response = await fetch(url);  
    const data = await response.json();  
    console.log(data);  
  } catch(error) {  
    console.log(error);  
  }  
}
```

Interview Questions

1. What is async and await in JavaScript?
2. Does async/await remove asynchronous behavior?
3. Can await be used outside async function?
4. What does async function return?
5. What is the difference between Promise and async/await?
6. How does await work internally?
7. Does await block JavaScript execution?
8. Why do we use try...catch with async/await?
9. What happens internally when await encounters a Promise?
10. Why is async/await preferred in enterprise applications?